

TCS34725 RGB Color Sensor

Hardware Overview

What is it?

The **TCS34725 Module** is an optical sensor that measures the composition of light. Unlike a simple light sensor (which only measures dark vs. bright), this chip contains a grid of microscopic photodiodes covered in physical red, green, and blue glass filters.

Electrical Explanation

The "Camera Shutter" (Integration Time): The sensor works like a digital camera. To measure color, it opens an internal electronic "shutter," collects light for a specific amount of time (e.g., 100 milliseconds), and then closes the shutter.

Active Sensing (The White LED): To read the color of a solid object (like a Lego brick), the sensor blasts the object with pure white light and measures what colors bounce back into the lens.

Key Hardware Facts:

- **Connection:** I²C Bus (SDA/SCL).
- **Default Address:** 0x29.
- **Voltage:** Operates at 3.3V to 5V.
- **Signal Flow:** Object → Bounced Light → RGB Filters → Analog-to-Digital Converter → I²C Bus → K-Comp Processor.

Software & Programming

kcomp_TCS34725.h

This library handles the I²C communication as well as a color distance algorithm for color estimation.

When you ask the library to estimate a color, it does not give you a word. It gives you a number which is assigned to a #define in the library. The following constants are defined:

COLOR_UNKNOWN, COLOR_BLACK, COLOR_WHITE, COLOR_GREY, COLOR_RED, COLOR_DARK_RED, COLOR_GREEN, COLOR_FOREST_GREEN, COLOR_LIME_GREEN, COLOR_BLUE, COLOR_NAVY_BLUE, COLOR_LIGHT_BLUE, COLOR_YELLOW, COLOR_LIGHT_YELLOW, COLOR_ORANGE, COLOR_PINK, COLOR_BROWN

Usage: #include <kcomp_TCS34725.h>

Function Reference

Function Name	Description & Parameters
<code>bool initColorSensor()</code>	Setup: Initializes the sensor. Returns <code>true</code> if successful, <code>false</code> if not found.
<code>void setColorIntegrationTime(uint8_t timeSetting)</code>	Setup: Changes shutter speed. Expects a <code>COLOR_TIME_...</code> macro.

	COLOR_TIME_2_4MS, COLOR_TIME_24MS, COLOR_TIME_50MS, COLOR_TIME_101MS (best for general scanning), COLOR_TIME_154MS, COLOR_TIME_700MS
void calibrateColorEngine(uint16_t blackThreshold, uint16_t whiteThreshold)	Setup: Sets the raw channel limits to catch noise and glare. 250 and 10000 might do.
void pollColorSensor()	The Worker: Must be called in loop(). Checks hardware and updates memory.
uint16_t getRed()	Returns the raw red count.
uint16_t getGreen()	Returns the raw green count.
uint16_t getBlue()	Returns the raw blue count.
uint16_t getClear()	Returns total unfiltered brightness. Used for calibration.
uint16_t getLux()	Calculates human-perceived Illuminance (Lux).
uint16_t getColorTemperature()	Calculates light temperature in Kelvin (K).
uint8_t estimateColor()	Runs the estimation algorithm. Returns a macro (e.g., COLOR_RED = 4).
const char* getColorName(uint8_t colorCode)	Translates the macro into an English string.

⚠ Pitfalls & Troubleshooting

The "Blind" Sensor (Distance)

If your color values are random or hovering near zero, the object is too far away.

The Fix: The object must be held about 5 to 15 mm above the black sensor lens.



Sample Program

This program initializes the sensor and prints some data.

```
#include <kcomp_TCS34725.h>

void setup() {
  Serial.begin(115200);

  if (!initColorSensor()) {
    Serial.println("Hardware Error: Sensor not found. Check wiring!");
    while(1); // Halt the processor
  }

  setColorIntegrationTime(COLOR_TIME_101MS);

  // STANDARD CALIBRATION: Black Threshold: 250 | White Threshold: 10000
  calibrateColorEngine(250, 10000);

  Serial.println("--- Sensor Initialized & Calibrated ---");
}

void loop() {
  // 1. Keep the internal state machine running constantly
  pollColorSensor();
  // 2. Trigger a scan every 2000 milliseconds (2 seconds)
  static unsigned long lastScan = 0;
  if (millis() - lastScan >= 2000) {
    lastScan = millis();
```

```
// Fetch and print all telemetry in a single line
Serial.print("RGB: ");
Serial.print(getRed()); Serial.print(", ");
Serial.print(getGreen()); Serial.print(", ");
Serial.print(getBlue());

Serial.print(" | Clear: "); Serial.print(getClear());
Serial.print(" | Lux: "); Serial.print(getLux());
Serial.print(" | Temp [K]: "); Serial.print(getColorTemperature());
Serial.print(" Est.: "); Serial.println(getColorName(estimateColor()));
}
```

💡 **Try it out:** You can find the full sample program **SimpleColorSensorTCS34725** in the K-Comp examples folder!